

Архитектура RISC-V

От однотоакового процессора к процессору конвеерного типа

Содержание (1/2)

- 1. Что происходит при выполнении любой инструкции
- 2. Общие этапы выполнения инструкции
- 3.1 Однотактный процессор. Упрощённый datapath
- 3.2 Однотактный процессор. Плюсы и минусы
- 4.1 Многотактный процессор
- 4.2 Многотактный процессор как автомат состояний
- 4.3 Многотактный процессор. FSM-представление
- 4.4 Многотактный процессор. Пример: addi
- 4.5 Многотактный процессор. Плюсы и минусы

Содержание (2/2)

- 4.6 Многотактный процессор. Сравнение с однотоковым
- 5.1 Конвейерный процессор
- 5.2 Конвейерный процессор. Классический 5-stage pipeline
- 5.3 Конвейерный процессор. Пример прохождения инструкций
- 5.4 Конвейерный процессор. Почему pipeline сложнее
- 6.1 Конфликты и их обход. Data hazard
- 6.2 Конфликты и их обход. Data hazard. Forwarding
- 6.3 Конфликты и их обход. Data hazard. Load-use hazard
- 6.4 Конфликты и их обход. Stall и bubble
- 6.5 Конфликты и их обход. Control hazard
- 6.6 Конфликты и их обход. Structural hazard
- Итоги

1. Что происходит при выполнении любой инструкции

Какая бы ни была инструкция, процессор обычно проходит примерно одни и те же шаги:

```
PC -> Instruction Memory -> Decode -> Register File -> ALU -> Data Memory -> Writeback
```

Для разных инструкций используются разные части этого пути:

addi — без памяти данных;

lw — с чтением из памяти;

sw — без записи результата в регистр;

beq — с проверкой условия и выбором нового PC;

jal — с записью PC+4 и переходом на новый адрес.

2. Общие этапы выполнения инструкции

Пример: `addi`

```
addi x5, x6, 10
```

Последовательность шагов:

1. прочитать инструкцию по адресу `PC` ;
2. декодировать `addi` ;
3. прочитать `x6` из Register File;
4. sign-extend для immediate `10` ;
5. ALU считает `x6 + 10` ;
6. результат записывается в `x5` ;
7. следующий `PC = PC + 4` .

3.1 Однотактный процессор. Упрощённый datapath

В однотактном процессоре инструкция проходит все этапы выполнения за **один такт**.

Примеры путей

Инструкция	Основной путь
<code>addi</code>	IMEM → RegFile → ALU → WB
<code>lw</code>	IMEM → RegFile → ALU → DMEM → WB
<code>sw</code>	IMEM → RegFile → ALU → DMEM
<code>beq</code>	IMEM → RegFile → Compare/ALU → PC logic

Самая длинная из этих цепочек определяет минимально возможный период такта

3.2 Однотактный процессор. Плюсы и минусы

Плюсы

- простая для понимания;
- базовый datapath;
- хорошо подходит как учебная модель.

Минусы

- период такта задаётся самой длинной инструкцией;
- быстрые инструкции ждут столько же, сколько `lw` или `branch` ;
- неэффективное использование аппаратуры;
- длинная комбинаторная цепочка плохо влияет на максимальную частоту;
- плохо масштабируются на практике, особенно на FPGA.
- на FPGA длинный критический путь мешает выйти на нужную частоту.

4.1 Многотактный процессор

В многотактном процессоре инструкция выполняется не за один длинный такт, а за несколько более коротких тактов.

одна инструкция = несколько тактов

Что это даёт

- длинный путь разбивается на несколько более коротких шагов;
- тактовую частоту можно сделать выше;
- одни и те же блоки можно использовать повторно в разные такты.

4.2 Многотактный процессор как автомат состояний

В многотактном процессоре управление удобно описывать как **автомат состояний**.

Каждый такт процессор находится в одном из состояний:

- **IF** — чтение инструкции;
- **ID** — декодирование и чтение регистров;
- **EX** — выполнение операции или вычисление адреса;
- **MEM** — доступ к памяти данных;
- **WB** — запись результата в регистр.

Инструкция проходит не весь путь сразу, а переходит из состояния в состояние.

При этом для разных инструкций путь по состояниям может отличаться.

4.3 Многотактный процессор. FSM-представление

Такой процессор очень естественно описывается как автомат состояний.

```
IF -> ID -> EX -> MEM -> WB
```

Путь зависит от инструкции:

- `add/addi` : IF -> ID -> EX -> WB
- `lw` : IF -> ID -> EX -> MEM -> WB
- `sw` : IF -> ID -> EX -> MEM
- `beq` : IF -> ID -> EX
- `jnl` : IF -> ID -> EX -> WB

4.4 Многотактный процессор

Пример: `addi`

```
addi x5, x6, 10
```

Такт	Состояние	Что происходит
1	IF	Чтение инструкции
2	ID	Декодирование, чтение <code>x6</code>
3	EX	Выполнение <code>x6 + 10</code>
4	WB	Запись результата в <code>x5</code>

Один и тот же ALU можно переиспользовать для разных инструкций на разных тактах.

4.5 Многотактный процессор. Плюсы и минусы

Плюсы

- критический путь короче, чем в однотоктном процессоре;
- проще выйти на более высокую частоту;
- аппаратные блоки можно использовать в разные такты повторно;
- разным инструкциям можно дать разное число тактов.

Минусы

- выполнение одной инструкции занимает несколько тактов;
- управление процессором становится сложнее;
- простота однотоктной схемы теряется.

4.6 Многотактный процессор. Сравнение с однотоактным

Условный пример для иллюстрации идеи.

Свойство	Однотоактный	Многотактный
Идея	1 инструкция за 1 такт	1 инструкция за несколько тактов
Длина такта	например, 10 нс	например, 2.5 нс
<code>addi</code>	1 такт = 10 нс	4 такта = 10 нс
<code>lw</code>	1 такт = 10 нс	5 тактов = 12.5 нс
Тактов на инструкцию	1	например, 4–5
Переиспользование блоков	слабое	хорошее
Частота	100 МГц	400 МГц
Простота объяснения	очень высокая	высокая

5.1 Конвейерный процессор

В multi-cycle одна инструкция проходит стадии **по очереди**.

В pipeline разные инструкции могут одновременно находиться на разных стадиях:

- одна в IF ;
- другая в ID ;
- третья в EX ;
- четвёртая в MEM ;
- пятая в WB .

Pipeline не ускоряет одну инструкцию

Pipeline повышает пропускную способность

5.2 Конвейерный процессор. Классический 5-stage pipeline

IF | ID | EX | MEM | WB

Между стадиями стоят pipeline registers:

IF -> IF/ID -> ID -> ID/EX -> EX -> EX/MEM -> MEM -> MEM/WB -> WB

Pipeline register — это набор регистров, который хранит:

- данные;
- номера регистров;
- immediate;
- управляющие сигналы.

5.3 Конвейерный процессор. Пример прохождения инструкций

```
addi x1, x0, 5  
addi x2, x0, 7  
add  x3, x1, x2
```

Такт	I1	I2	I3
1	IF		
2	ID	IF	
3	EX	ID	IF
4	MEM	EX	ID
5	WB	MEM	EX
6		WB	MEM
7			WB

5.4 Конвейерный процессор. Почему pipeline сложнее

Как только инструкции начинают идти параллельно, они начинают влиять друг на друга.

Появляются три типа проблем:

- **конфликт по данным** — данные ещё не готовы;
- **конфликт по управлению** — следующий РС ещё не ясен;
- **структурный конфликт** — один ресурс хотят использовать сразу две стадии.

Именно из-за этого pipeline мощнее, но сложнее.

6.1 Конфликты и их обход. Data hazard

Пример:

```
add x5, x1, x2  
sub x6, x5, x3
```

Проблема:

- `sub` хочет использовать `x5` ;
- но `add` ещё не успел записать его в Register File.

Если ничего не делать,
следующая инструкция увидит старое значение.

6.2 Конфликты и их обход. Data hazard. Forwarding

Решение для многих data hazard:

не ждать записи в регистр,
а подать результат напрямую из более поздней стадии pipeline.

Идея bypass-путей:

```
EX/MEM result -> ALU input  
MEM/WB result -> ALU input
```

То есть ALU следующей инструкции получает уже готовый результат, не дожидаясь обычной записи результата.

6.3 Конфликты и их обход. Data hazard. Load-use hazard

Пример:

```
lw  x5, 0(x1)
add x6, x5, x2
```

Почему тут сложнее:

- результат `lw` появляется только после доступа к памяти;
- следующая инструкция хочет использовать `x5` уже на своей стадии `EX`.

Обычно одного forwarding недостаточно.

Нужен **stall** — вставка пузыря (**bubble**) в pipeline.

6.4 Конфликты и их обход. Stall и bubble

```
lw    x5, 0(x1)
add   x6, x5, x2
```

Такт	lw x5, 0(x1)	add x6, x5, x2
1	IF	
2	ID	IF
3	EX	ID
4	MEM	stall
5	WB	EX
6		MEM
7		WB

Смысл stall(задержки):

- мы временно задерживаем инструкцию в определенной части конвейера, чтобы не использовать неготовые данные
- чтобы в конвейере не было мусора используем bubble (NOP) или нулевые управляющие сигналы

6.5 Конфликты и их обход. Control hazard

Пример:

```
beq x1, x2, target
```

Проблема:

- пока branch не вычислен, неясно, какой будет следующий РС ;
- процессор может уже начать fetch по неверному пути.

Типичное решение в простом pipeline:

- дождаться результата branch;
- если пошли не туда — сделать **очистку** ошибочно загруженных инструкций.

6.6 Конфликты и их обход. Structural hazard

Structural hazard возникает, если две стадии хотят использовать один и тот же ресурс одновременно.

Пример:

- стадия IF хочет читать инструкцию;
- стадия MEM хочет читать или писать данные;
- а память одна.

Возможные решения:

- отдельная память для инструкций и данных;
- арбитраж;
- stall.

Итоги

Мы увидели, что одна и та же инструкция RISC-V может выполняться по-разному:

- в **однотактном процессоре** — целиком за один такт;
- в **многотактном** — по шагам за несколько тактов;
- в **конвейерном** — с перекрытием выполнения разных инструкций.

Чем эффективнее организация выполнения,
тем сложнее становится управление процессором.

Запомнить

ISA задаёт, что должна делать инструкция,
а микроархитектура определяет, как именно процессор это реализует.